A Study Report On

Monarch: Google's Planet-Scale In-Memory Time Series Database

By

Datta Ksheeraj

21CS30037

Department of Computer Science and Engineering

Indian Institute of Technology Kharagpur

Autumn Semester, 2024-25

Contents

List of Figures

Chapter 1

Introduction

1.1 Basics

Google operates one of the largest and most complex infrastructures in the world, with millions of servers across data centers worldwide. Monitoring and managing performance metrics across this massive infrastructure requires a database capable of handling petabytes of data and millions of queries per second.

Google uses various types of databases, each suited for specific tasks. Examples include Spanner, Bigtable, Colossus, BlobStore etc. There should be monitoring system over all these database systems. Generally a TSDB (Time-Series Database monitoring system is used which captures events along with time stamps

1.2 Monarch as TSDB

A time-series database (TSDB) is a specialized database optimized to handle, store, and analyze time-series data, which consists of data points indexed by time. This

type of data typically represents how a system, process, or measurement evolves over time, with each entry or observation paired with a timestamp

At Google, the time-series database (TSDB) landscape has evolved significantly over the years, primarily driven by internal projects and open-source contributions to handle the massive scale of data generated by Google's infrastructure

The initial TSDB developed and used was Borgmon which was replaced with Monarch for reasons discussed later in the report

So currently Monarch is the globally-distributed in-memory time series database monitoring system in Google

Chapter 2

The Problem

2.1 Arrival of Borgmon

2.1.1 Initial Problem

Google's infrastructure comprised numerous clusters, each hosting thousands of servers running various applications. The complexity and scale of these clusters made it challenging to monitor system health effectively. Existing monitoring solutions lacked the ability to provide real-time insights into the status of individual servers, services, and applications. This often resulted in:

- Need to get a time-based collection of data.
- Difficulty in identifying failing or underperforming nodes.
- Slow response times to incidents due to limited visibility.
- Overwhelming volumes of alerts, many of which were not actionable.

2.1.2 Borgmon Solution[1]

Borgmon was designed as a time-series database to provide comprehensive cluster monitoring by implementing:

- **Real-time Monitoring:** Borgmon enabled real-time tracking of system health, resource usage, and performance metrics across all nodes in a cluster.
- **Centralized Data Collection:** It centralized data collection from all servers, allowing for a unified view of the health of the entire infrastructure.
- **Intelligent Alerting:** The system incorporated intelligent alerting mechanisms that reduced noise by prioritizing alerts based on severity and context, thus allowing engineers to focus on critical issues.

2.2 Arrival of Monarch[2]

Google shifted from Borgmon to Monarch to address the growing complexity and scale of its monitoring needs, especially as its infrastructure became more distributed and global. Below are the key reasons behind this shift:

- **Scalability and Performance**
  - **Borgmon Limitations:**
    - Borgmon was originally designed to monitor Borg, Google's cluster manager, primarily handling a centralized monitoring model.
    - As Google's infrastructure expanded, including global data centers and cloud-based services, Borgmon struggled to scale effectively also making it difficult for engineers to run Borgmon reliably.
- **High Availability and Fault Tolerance**

  - **Redundancy and Failover:**
    * Borgmon, while robust for centralized environments, lacked the same level of built-in redundancy for global distributed systems.

- **Efficient Data Aggregation and Querying**
  - **Advanced Aggregation Techniques:**
    * Borgmon had fewer optimizations for data aggregation and was limited in handling high cardinality metrics efficiently.

- **Cost Optimization**
  - **Storage Efficiency:**
    * Borgmon's design was less optimized for the vast volumes of data generated in real-time monitoring, leading to higher storage and processing costs.
  - **User-Configurable Replication Policies:**
    * Monarch allows Google to set variable replication levels based on the needs of different services, balancing availability with storage cost.
    * This flexibility was not available in Borgmon, which had a more one-size-fits-all approach.

- **Supporting Modern Monitoring Needs**
  - **High Cardinality and Complex Data Types:**
    * Borgmon was limited in handling high-cardinality metrics, which became increasingly necessary as Google's services and user interactions grew.
  - **Dynamic Load Balancing:**
    * Borgmon lacked adaptive load-balancing features, making it harder to maintain performance during fluctuating workloads.

Chapter 3

Motivation

3.1 Core Solution Ideas

Keeping in mind the various problems or limitations with Borgmon mentioned in Chapter2, the solutions proposed for each are trivial. This slowly led to the foundation of Monarch, the now widely used replacement of Borgmon.

The solutions for each of the problems were as mentioned below:

- **Scalability and Performance:** The system instead of having a centralized architecture, now should have a decentralized one. This now adds on complexity to development of such architecture hence is a good trade off (Definitely if putting more effort brings in a system which is more scalable Why not?)
- **Fault Tolerance:** Simple solution for this is to maintain replicas which is indeed done in Monarch. Also to improve performance we maintain each replica to a different level of granularity (detailing).
- **Storage Issue:** With petabytes of data needed to be stored we need to introduce some sort of compression or new data structures to hold data effectively

and in cheapest possible manner. For some encoding techniques were used which serve the purpose.

- **Load Balancing:** Again this is a very important performance optimizing technique which can bring down performance by large scale if not implemented properly. In Monarch this is done as part of ingestion of data and can also be done individually by each local storage unit as we shall see later.

With all the solutions looking clear, the actual challenge lies in putting them all together which brings out a system better than Borgmon. As we shall see Monarch had successfully achieved that.

Chapter 4

Implementation

4.1 Components

Monarch's architecture includes components categorized by their primary functions: those responsible for storing state, those dedicated to data ingestion, and those focused on query execution.

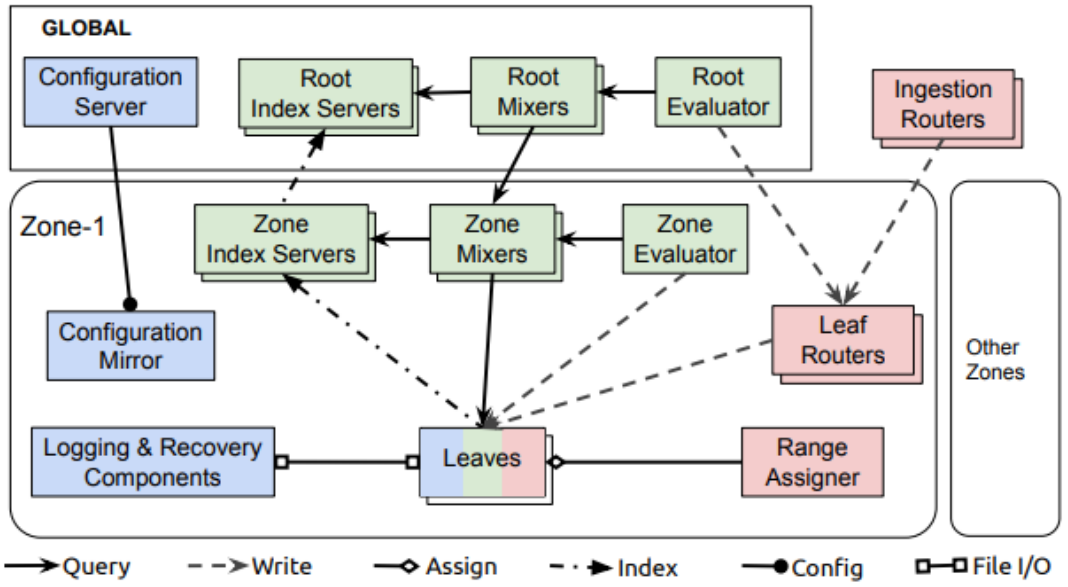

FIGURE 4.1: Overview of Monarch [2]

State-Holding Components

- **Leaves** serve as the primary storage for monitoring data, keeping it in an in-memory time series database.
- **Recovery logs** store monitoring data on disk, mirroring the data held in leaves. Over time, this data is moved into a long-term time series repository.
- **Global configuration server and zonal mirrors** manage configuration data, which is stored in Spanner databases [2].

Data Ingestion Components

- **Ingestion routers** direct data to leaf routers within the correct Monarch zone, relying on information embedded in time series keys for accurate routing.
- **Leaf routers** receive data bound for a specific zone and forward it to the leaves for storage.
- **Range assigners** handle the distribution of data across leaves, balancing the load within each zone.

Query Execution Components

- **Mixers** divide queries into smaller sub-queries routed to and processed by leaves, then combine the results. Queries can be initiated at the root level (handled by root mixers) or at the zone level (handled by zone mixers). Root-level queries require both root and zone mixers.
- **Index servers** provide indexing for each zone and leaf, guiding the distributed execution of queries.
- **Evaluators** periodically execute standing queries (see Section 5.2) by dispatching them to mixers, then write the results back to leaves.

4.2 Data Storage

Monarch organizes monitoring data into tables that store time series, which are sequences of values recorded over time. Each table includes two main types of columns:

- **Key columns:** These define unique identifiers for each time series, such as the specific machine or service being tracked.
- **Value column:** This stores the historical data points (e.g., memory usage or latency) recorded over time for each time series.

Key columns, also referred to as fields, have two sources: targets and metrics.

Targets (Monitored Entities)

- A target is the system or component that generates data, like a server, application, or process.
- Monarch links each time series to its specific source target, identifying the entity from which the data originates.
- Each target follows a schema (template) with specific fields, such as user, job, cluster, or task number for tasks in a computing cluster.
- The location field within each target helps Monarch store data near its origin (for instance, data from a specific cluster is stored in the closest zone).
- Time series data from the same target are grouped, ensuring related data is stored together and is easier to query.
- Target ranges organize targets in alphabetical order to allow efficient data distribution and load balancing, which simplifies group queries.

Metrics (What We're Measuring)

- A metric is a specific measurement related to a target, such as the number of requests a server processes or its memory usage.
- Each metric has its own schema with particular fields, such as the type of service or command for an RPC request.
- Metrics can be various data types, including numbers, strings, or distributions (groups of values that allow for calculations of averages, percentiles, etc.).

4.3 How Data Storage Happens

The data collection path in Monarch can be broken down into four main steps:

- **Client to Ingestion Router**
  - A client application sends time series data to a nearby ingestion router. Monarch's ingestion routers are spread across clusters, allowing data to be gathered close to its origin, thus reducing latency.
  - Clients frequently utilize Monarch's instrumentation library, which regulates the data write frequency to comply with Monarch's retention policies.
- **Ingestion Router to Leaf Router**
  - Each ingestion router extracts the location field of the target to determine the appropriate zone for the data, organizing it by geographic or logical regions.
  - After identifying the target zone, the ingestion router forwards the data to a leaf router within that zone. Zone mappings are dynamically updated.

- **Leaf Router to Leaf Node**
  - Within the assigned zone, the leaf router directs data to specific leaf nodes. These leaf nodes manage target ranges, with data sharded lexicographically according to target strings.
  - Each leaf router keeps an updated range map that indicates which leaf node is responsible for each target range. This map is periodically refreshed by updates from leaf nodes, ensuring uninterrupted data collection even if the range assigner experiences temporary issues.
- **Data Storage on Leaf Nodes**
  - Upon reaching the assigned leaf node, data is stored in an in-memory time series store and recovery logs. This in-memory store is optimized for efficient handling of timestamp sequences, reducing memory usage.
  - Data encoding methods, such as **delta encoding** and **run-length encoding**, are applied to efficiently manage various time series types, including complex structures like tuples.

Chapter 5

Organization of Data and Load Balancing

5.1 Data Organization and Load Balance

Intra-zone load balancing in Monarch is a strategy used to ensure efficient data storage and distribution across leaf nodes within a zone, minimizing load and optimizing performance. Here's an in-depth look at how this process works:

- **Schema and Lexicographic Sharding**
  - **Data Organization:** Data is organized in a table schema consisting of a target schema and a metric schema.
  - **Sharding by Target:** Only the key columns associated with the target schema are used for sharding data lexicographically. This approach groups all time series for a single target together, which minimizes the ingestion fanout, allowing a single message to carry data for multiple metrics for the same target. As a result:
    * Each target's data is only sent to a few (up to three) leaf replicas.

    - This setup scales the zone horizontally by adding more leaf nodes and simplifies query processing by limiting it to a smaller subset of leaf nodes.

  - **Intra-Target Joins:** Common joins between metrics for the same target can be processed within the leaf node, reducing query complexity and making them faster.

- **Replication Flexibility**

  - **Replication Policy:** Monarch allows users to select the number of replicas per target (ranging from 1 to 3), enabling a trade-off between availability and storage cost.
  - **Granularity Control:** Users can choose to retain different levels of data detail for each replica, such as retaining only a fine-grained view on one replica and a coarser view on others.
  - **Individual Assignment of Replicas:** Each target range replica is assigned independently to avoid overloading a single leaf node, ensuring no leaf node holds multiple replicas of the same range.
  - **Distribution Across Failure Domains:** Leaves are distributed across clusters or failure domains. The range assigner ensures replicas of the same range are stored in different domains to enhance fault tolerance.

- **Range Assigner and Load Balancing**

  - **Load Monitoring and Adjustment:** The range assigner actively balances the load by moving, splitting, or merging ranges as needed based on CPU load and memory usage across leaf nodes.

- **Ensuring Data Availability**

  - **Simultaneous Data Collection:** During the transfer process, both the source and destination leaves temporarily collect and log data for range

R to avoid any data loss and ensure continuous availability.(**Interesting optimisation**)

- **Direct Updates from Leaves:** Leaves keep leaf routers informed about range assignments, rather than relying on the range assigner for updates(**Interesting optimisation**). This:
  - Ensures data integrity, as leaves are the main storage units.
  - Allows the system to continue working smoothly if the range assigner has a temporary failure.

Chapter 6

Querying

6.1 Query Execution

In Monarch, there are two types of queries:

Ad hoc Queries These are one-time queries from users outside the system.

Standing Queries These are periodic queries whose results are stored back in Monarch. Standing queries are used to generate alerts or pre-process data for faster and more cost-effective access later. They can be processed at either the regional zone level or the global root level, based on the type of data and query requirements. Most standing queries are handled at the zone level, making them more efficient and resilient to network issues.

6.1.1 Query Tree and Execution Levels

Queries are processed in a three-level hierarchy:

- **Root Mixer:** Receives the query and distributes it to zone mixers.

- **Zone Mixers:** Send the query to relevant leaf nodes based on an index.
- **Leaf Nodes:** Process the data closest to the source.

Each query is executed only at the necessary levels, which optimizes processing. The root node checks security, access control, and can rewrite queries for efficiency. Lower levels (leaves and mixers) stream data upwards, where higher levels combine results and manage the flow of data with rate control.

6.1.2 Replica Resolution

Data is often replicated, and replicas may vary in quality (e.g., completeness and time coverage). For accurate results, zone mixers resolve which replica is best based on quality criteria and assign target ranges to specific leaves for processing.

6.1.3 User Isolation

Monarch is a shared system, so resources are divided among users. Each user's queries are limited in memory and CPU through cgroups, ensuring fair use.

6.1.4 Query Pushdown

Monarch minimizes data transfer by processing as much of the query as possible at the lowest level. This is called "query pushdown" and improves speed by:

- Reducing the amount of data transferred to higher levels.
- Allowing more concurrent processing.

For example, if a query only involves data from one zone, it can be fully processed there without needing root-level involvement. This pushdown approach makes 95% of standing queries complete within the zone, which also prevents cross-region data traffic and reduces latency.

6.1.5 Specific Operations at Lower Levels

Some queries, like `group by` or `join`, are pushed to the leaf level when possible, where they process data within the smallest target ranges. For instance, if data for a specific target (like a task) remains within one leaf, that leaf can complete the operation, which reduces work for higher-level nodes.

6.2 Result Aggregation

- Leaf Nodes process data closest to the source. They handle their assigned "target range" and perform basic filtering and aggregation to reduce data before sending it up.
- Zone Mixers gather results from multiple leaf nodes in their region. They combine and further aggregate data, then send only summarized information to the root mixer.
- Root Mixer collects data from zone mixers. It completes any remaining aggregation, checks security, and applies final optimizations.

Chapter 7

How Monarch solved previous Limitations

7.1 Looking Back

- **Scalability and Performance**:
  - Monarch's architecture, as seen in implementation part 4, is a decentralized global system that enables independent control over various parts, making it convenient for engineers to operate and debug.
  - For high scalability, Monarch uses a field hints index (FHI), stored in index servers, to limit the load when sending a query from parent to children by skipping irrelevant children (those without input data to the particular query). This is implemented using n-grams for the data and storing what all leaves contain that.
- **Fault Tolerance**:
  - Users have the option to maintain replicas of data in whatever number and granularity they need, as stated in previous chapters. Depending on

these parameters, performance is traded off since the detailing and search time of each replica differs.

  - The implementation of this feature led developers to introduce new components, like range assigners, which play a key role in maintaining fault tolerance. It also introduces various factors to consider, such as where the replicas are stored and how to maintain consistency.

- **Storage**: For the vast incoming data which is at rate of petabytes per unit time, Monarch used delta-time encoding and other techniques as mentioned in Chapter 4 and Chapter 5.

- **Load Balancing**: This was implemented and had enhanced the system's performance a lot. The detailing was given in Chapter 5. This was a very interesting optimization used in many systems as well.

Overall, though individual solutions were trivial, integrating them into a single system takes lot of effort thus the complex architecture of Monarch. This was clearly a trade off between performance and the hardware or the architecture, which in this case was more towards architecture.

Monarch prioritizes high availability and partition tolerance over consistency. Interactions with strongly consistent databases, such as Spanner, can result in lengthy blocks, which are not suitable for Monarch. This blocking could prolong the meantime-to-detection and mean-time-to-mitigation during potential outages. To ensure timely alert delivery, Monarch focuses on providing the most recent data quickly. As a result, it may drop delayed writes and, when necessary, return partial results for queries.[2]

Chapter 8

My Observations

8.1 My Observations and Possible Solutions

While Monarch prioritizes high availability and partition tolerance, the trade-off with data consistency could lead to stale reads. This seemed more bold decision to me, instead we should be using better consistency model which does different things based on situation.

For this I thought of implementing a Reinforcement Learning Model which slowly learns the data it handles and performs accordingly.

Also user was given choice to choose the number of replicas and level of granularity. But when scalability is considered and lot of users operate at same time they might over estimate the usage which unnecessarily creates extra load if all created maximum number of replicas.

For this the control given to users must be looked into again, there must be control mechanisms asking users about the data they are going to habdle and based on that the system should have some percentage of controlling. Even complete control to the system itself is a waste.

Bibliography

[1] Borgmontsdb. https://sre.google/sre-book/practical-alerting/.

[2] Colin Adams, Luis Alonso, Ben Atkin, John P. Banning, Sumeer Bhola, Rick Buskens, Ming Chen, Xi Chen, Yoo Chung, Qin Jia, Nick Sakharov, George T. Talbot, Adam Jacob Tart, and Nick Taylor, editors. *Monarch: Google's Planet-Scale In-Memory Time Series Database*, 2020.